agent_usage 계산기 해설 — 구조, 기록 데이터, 그리고 손댈 수 있는 곳

2026-11-05 기준 스냅숏 · 기록 362건(끝난 것 352건, 마지막 p237) · 지시서 p237(배치 69) · 숫자는 모두 이날 명령으로 다시 뽑았다

요약

이 글은 하위 에이전트의 걸리는 시간·턴·토큰을 보내기 전에 내다보는 '계산기'(`domains/agent_usage/predict.py`)가 어떻게 짜여 있고, 그것이 무엇을 재료로 삼으며, 어디서 왜 틀리는지를 설명한다. 계산기는 보내기 전에 답할 수 있는 예/아니오 질문(상태) 30개 남짓에 대한 답과 저장소 이름만 보고, 효과가 붙은 가설(조각) 17개를 곱해 범위를 내고, 그 범위의 기하 가운데 값이 떨어지는 시간 구간 하나를 사용자에게 말한다. 조각의 숫자는 2026-10-19까지의 기록 126건으로 한 번 맞춘 뒤 고정되어 있고, 그 뒤 226건에서 계산기는 말할 구간을 53건(23 %, 그중 61건은 답 자체를 못 냄) 맞혔다. 같은 기록에서 메인이 손으로 한 예측은 148건(65 %)을 맞혔다. 일의 크기를 직접 세는 칸(쓰는 파일 수 `files`)은 11-04에 생겼고, 보내기 전에 센 끝난 기록은 25건뿐이며, 그 칸을 쓰는 비율 조각은 시험 4건에서 모두 지나쳤다. 그러므로 "정해진 일이고 크기도 적는데 왜 어려운가"에 대한 짧은 답은, 새 종류의 데이터가 아직 너무 적고(두 저장소 25건), 계산기의 구조가 그 수를 거의 쓰지 않거나(스위치 경로) 비례로만 쓴다(비유 경로)는 것이다. 끝 절은 탐색할 질문, 구조를 바꿀 자리, 새로 모을 칸을 근거의 강도와 함께 늘어놓는다.

1. 문제 설정 — 무엇을 누가 무엇으로 판정하나
2. 데이터 — 기록 한 건의 칸과 그 내력
3. 계산기의 구조 — 조각, 조합, 말할 구간
4. 맞추기 — 숫자를 기록에서 정하는 법
5. 시험과 장치 — 사후 설명을 막는 규칙들
6. 주변 모델 — 손 규칙, 빗댄 기록, 크기, 비율, 보낼까
7. 현재 성적과 오늘 확인한 것
8. 개입 지점 — 탐색 질문, 구조, 수집
부록 A. 문서와 코드가 어긋난 곳
부록 B. 용어 정리

읽는 법. 본문의 숫자는 2026-11-05에 저장소 루트에서 돌린 명령과 읽기 전용 스크립트의 출력이고, 숫자마다 어느 명령에서 나왔는지는 보고서 `docs/worklog/2026-11-05-calculator-explainer.md`에 적었다. 성격이 다른 숫자에는 꼬리표를 붙인다. 시험은 숫자나 규칙을 세운 뒤에 나온 기록으로 잰 것, 맞춰 봄은 세울 때 본 기록으로 잰 것, 사후는 이번에 데이터를 보고 고른 비교라 판정이 아닌 것, 짐작은 확인하지 않은 해석이다. 앞선 분석(2026-11-04 p179 보고서 등)을 옮길 때는 날짜와 출처를 붙인다.

---

1. 문제 설정

1.1 예측하는 것: 분, 턴, 토큰

이 분야가 예측하는 대상은 메인 세션, 곧 사용자와 대화하는 모델이 지시서 한 장을 써서 하위 에이전트에게 일을 맡겼을 때 그 일이 끝나기까지의 세 수량이다. 한 번 맡긴 일을 '보냄'(기록의 `[[dispatch]]` 하나)이라 하고, 같은 에이전트가 앞 일을 마친 뒤 이어서 다시 받은 일을 '회차'라 하며, 한 번에 함께 내보낸 보냄들의 묶음을 '배치'라 부른다. 세 수량 가운데 사용자에게 직접 의미가 있는 것은 벽시계 분이고, 턴과 토큰은 분을 설명하는 중간 변수이자 비용의 척도다.

실측은 사람이 아니라 스크립트가 한다. `scripts/agent_usage.py --actual`은 하위 에이전트의 대화 기록(JSONL)을 읽어, 첫 줄과 마지막 줄의 시각 차이를 분으로, 모델이 남긴 줄의 수를 턴으로, 줄마다 기록된 입력·캐시 생성·캐시 읽기 토큰에 출력 추정치(글자 수 ÷ 3.6)를 더한 값을 토큰으로 센다. 완료 알림에 실행 시간이 찍혀 있으면 분은 그것을 쓴다. 여기서 주의할 것은 '턴'의 단위다. 모델의 응답 하나는 생각 블록·글 블록·도구 호출 블록마다 따로 한 줄씩 남으므로, 기록의 턴은 응답이 아니라 **블록**의 수다. 2026-11-03에 p145 기록으로 확인한 바로는 28줄이 응답 15개였고, 같은 응답의 줄마다 사용량이 반복 기록되어 줄마다 더한 토큰(3.16M)이 응답마다 한 번 센 값(1.72M)의

1.83배였다. 사용자는 11-04에 이 단위를 그대로 두기로 했다("응답 단위는 우리가 맞히려는 거고"). 지난 기록이 모두 같은 방식으로 재어졌으므로 기록끼리의 비교는 공정하지만, 분당 턴은 '분당 블록'이고 토큰은 응답 단위의 약 두 배라는 점을 늘 염두에 두어야 한다.

1.2 사용자에게 말하는 것: 세 구간 중 하나

사용자가 받는 것은 숫자 범위가 아니라 행동으로 번역된 한마디다. 실제 분이 10분 이하면 '잠깐'("잠깐 기다려"), 10분 초과 40분 이하면 '다른 일'("다른 거 하고 있어"), 40분 초과면 '신경 끄기'("신경 쓰지 말고 있어")라 하고, 이 셋을 이 글에서 '시간 구간'이라 부른다(`dispatches.zone`). 예측은 분 범위 $[a,\ b]$로 잡지만 사용자에게는 그 범위의 기하 가운데 값이 떨어지는 구간 하나를 말하며, 이를 '말할 구간'이라 한다.

$$\text{말할 구간} = \text{zone}\left(\sqrt{a \cdot b}\right), \quad \text{zone}(m) = \begin{cases} \text{잠깐} & (m \le 10) \\ \text{다른 일} & (10 < m \le 40) \\ \text{신경 끄기} & (m > 40) \end{cases} \tag{1}$$

예컨대 6–15분이라는 예측은 가운데 값 9.5분이므로 '잠깐'으로 말한다. 이 규칙(R_say_numbers_v3)은 2026-11-03부터이고, 그 전(v2)에는 범위가 두 구간에 걸치면 긴 쪽을 말했다. 바꾼 근거는 지난 손 예측 256건에 두 규칙을 대보았을 때 긴 쪽이 67 %, 가운데 값이 78 %를 맞혔다는 사후 비교였다. 그 뒤로 v3로 말한 시험 기록 66건 중 60건이 맞았다 [시험]. 이 '말할 구간 적중'이 모든 성적표의 맨 윗줄이며, 분 범위가 실측을 담았는지(분 범위 적중)는 그보다 엄격한 둘째 잣대다.

1.3 누가 예측하고 무엇이 판정하나

한 보냄에는 성격이 다른 예측이 최대 셋 붙는다. 첫째는 '손 예측'(`predicted`)으로, 메인이 지시서를 보고 비슷한 지난 일과 손 규칙(R_*)을 참고해 보내기 전에 적는 분·턴·토큰 범위와 말한 구간이다. 둘째는 '계산기 답'(`calc`)으로, 같은 기록의 상태 값을 넣어 `predict.py`를 돌린 결과다. 셋째는 시험 중인 '빗댄 기록 규칙'(`analog.py`)으로, 정해진 일이면 가장 가까운 지난 기록을 기계가 골라 그 시간을 따른다. 사용자는 이 분야의 예측을 판단하지 않기로 했으므로(2026-10-16 결정) 판정자는 실측뿐이고, 돌아보기에서 까닭을 찾지 못한 빗나감('까닭 모름')만 사용자에게 넘긴다. 손 예측과 계산기 답을 늘 함께 적게 한 것(2026-10-19 결정)은 어느 쪽이 나은지를 같은 기록에서 계속 세기 위해서다.

---

2. 데이터

2.1 단위와 규모

데이터의 단위는 `domains/agent_usage/dispatches.toml`의 `[[dispatch]]` 하나이고, 한 건은 예측 → 실측 → 돌아보기의 순서로 채워진다. 기록은 보낸 순서대로 파일 끝에 덧붙이며, 이미 적힌 예측·실측 숫자는 고치지 않는다는 규칙이 있다. 메인이 하위 에이전트를 보내지 않고 직접 한 일은 같은 모양으로 `direct.toml`에 따로 적는데, 지금 1건(m001)뿐이다. 2026-11-05 현재 보냄 기록은 362건이고, 그중 끝까지 한 것(`status = "done"`, 실측 분 있음)이 352건, 중간에 멈춘 것 6건, 끝났는지 모르는 것 4건이다. 성적은 끝난 352건으로만 매긴다.

표 1. 기록의 출처와 저장소. id의 앞 글자는 '어느 세션이 적었나'이고 저장소(project)는 '어느 일인가'다 — p로 시작해도 다른 저장소의 일일 수 있다.

구분	건수	설명
d·dR (batman 지휘 모델)	125	d 117 + dR 8. 다른 저장소(batman)의 지휘 모델이 적은 예측 줄을 옮겨 온 것. 날짜 10-13–10-18. 조각의 숫자를 맞춘 p05까지 126건 중 121건이 여기다
p (이 저장소의 세션)	237	p01–p237. 저장소별: proviso 61 · movie_game 38 · gemmin 36 · exam_grader 32 · language_learning 23 · suneung 22 · professor_king 13 · stock-rader 7 · font 5
상태(끝난 것 / 멈춤 / 모름)	352 / 6 / 4	멈춤·모름은 판정에서 빠진다

표 2. 모델·effort(`dispatches.setup`으로 푼 값). 기본 설정은 Opus 5.5 high이고, 칸이 비어 있는 옛 기록 34건은 '기본 설정 아님'이 아니면 기본 설정으로 푼다.

설정	건수	설정	건수	설정	건수
Opus 5.5 high	270	Haiku 5.5 (effort 모름)	25	Opus 5.5 xhigh	15
Haiku 5.5 high	12	Haiku 5.5 medium	12	Sonnet 5.5 max	11
Fable 5.1 xhigh	11	Opus 5.5 medium	5	Fable 5.1 (effort 모름)	1

기간을 넷으로 나누어 보면 데이터의 성격이 시간에 따라 크게 바뀌었음이 드러난다(표 3). 초기 기록은 batman의 큰 구현이 중심이라 '신경 끄기'가 흔했고, 11-04 이후에는 같은 에이전트가 이어 쓰는 1–3분짜리 풀이지 일이 많아 실제 구간의 71 %가 '잠깐'이다. 기준선도 함께 바뀐다는 뜻이다. 늘 '잠깐'이라고만 말해도 11-04 이후 55건에서 39건(71 %)을 맞히므로, 이 시기의 적중률을 읽을 때는 이 기준선과 견주어야 한다.

표 3. 기간별 끝난 기록과 실제 시간 구간. '정해진 일'은 S_specified = 예(만들 것과 고칠 곳이 지시서에 다 적힘)인 기록.

기간	끝난 기록	잠깐	다른 일	신경 끄기	정해진 일	주된 일
10-13 – 10-27	218	68	102	48	127	batman 구현·조사, movie_game, language_learning
10-28 – 11-02	45	22	22	1	33	수능 지문 분석, 풀이지 일
11-03	34	3	30	1	10	먼 분야 경험칙 생성(같은 지시서 28건)
11-04 – 11-05	55	39	16	0	30	exam_grader 풀이지 시리즈, gemmin 정리·조사

2.2 칸의 무리와 각 칸의 내력

한 기록의 칸은 여섯 무리로 나뉜다. 머리 칸(`id`·`project`·`batch`·`date`·`task`, 나중에 더해진 `model`·`effort`·`agent`·`round`), 보내기 전에 적는 상황 값(`[dispatch.situation]`)과 일의 크기(`size`·`files`), 두 예측(`predicted`·`calc`)과 보낼까 판단(`decide`), 실측(`actual`)과 턴 길이 표(`turns`), 그리고 돌아보기(`review`)다. 데이터를 분석하는 사람에게 중요한 것은 칸의 뜻보다 그 칸이 **언제부터, 누가, 결과를 보기 전에 채웠는가**이다. 같은 칸이라도 보내기 전에 적은 값과 결과를 본 뒤 채운 값은 통계적으로 다른 변수이기 때문이다. 표 4는 이를 칸마다 정리한 것이다.

표 4. 칸별 있음 수(전체 362 / 끝난 352), 처음 나온 날, 검사가 필수로 막기 시작한 날, 채운 방식. 필수 날짜는 `dispatches.py`의 상수에서 읽었다.

칸	뜻	있음	처음	필수(검사)	채운 방식·주의
`situation`	상태마다 예/아니오/일부/모름	362/352	10-13	—	기록당 적힌 상태 중앙 20개(2–26). 여러 상태를 10-16·10-19·10-23·11-03에 사후 채움
`predicted.minutes`	손 예측 분 [하한, 상한]	355/346	10-13	—	보내기 전이 아닌 것 11건(`written_before = false`, 첫 배치 등)
`predicted.turns` · `tokens_m`	손 예측 턴·토큰(백만)	326 · 265	10-13 · 10-18	—	토큰은 처음엔 등급(T1–T4)만
`predicted.told`	사용자에게 말한 구간	234/228	10-19(p04)	—	11-03 전에는 걸치면 긴 쪽(v2)으로 말한 값
`predicted.analog`	메인이 고른 빗댄 기록 id	190/184	10-19(p04)	—	AI의 선택이라 재현되지 않는다. 비유 계산의 입력
`predicted.rules`	손 예측에 쓴 규칙 id	362	10-13	id 검사만	규칙의 '쓴 예측' 성적이 이것으로 매겨진다
`calc`	보낼 때 돌린 계산기의 구간·분	167/162	10-20(p06)	—	판(조각 버전)·종류(스위치/비유)가 적히지 않아 섞여 있다(p179 보고서 C3)
`actual` 분·턴·토큰	실측	356 · 313 · 352	10-13	끝난 기록에 분	턴 없는 끝난 기록 43건(클라우드 세션 41, batman 2)
`actual.start/end`	실측 시작·끝 시각(UTC)	69/68	11-03(p145)	11-04	빗댄 기록 규칙이 '보낼 때 끝나 있던 기록'을 가르는 데 쓴다
`agent` · `round`	같은 에이전트 이름, 몇 번째 회차	72/68	11-03(p145)	11-04	이름 54개. 가장 큰 묶음은 exam_grader 한 에이전트의 12회차(14–25회차)

칸	뜻	있음	처음	필수(검사)	채운 방식·주의
`model` · `effort`	보낸 모델과 effort	328 · 302	10-16	11-04	지난 261건은 11-03에 작업 설명에서 기계로 채움. S_other_model은 이 둘에서 계산
`size`	지시서에서 센 수(읽을 것·만들 것·덩어리·고칠 곳·확인)	316/309	(11-03 사후)	모양만	결과를 가린 하위 에이전트가 작업 설명 요약만 보고 269건을 채움. 단위가 섞임(1–1,350)
`files`	쓰는 파일 수 {write, edit}	59/55	11-04	11-04	빈 표(모름) 18, 결과 뒤 채움 `after` 11. 보내기 전에 센 0 아닌 끝난 기록 25
`decide`	보낼까 판단(두 쪽 토큰 추정과 까닭)	69/65	11-03(p169)	11-04	69건 모두 '보냄'
`turns`	동작별 턴 길이 표	99/96	11-02(p138)	11-03	표 74, 못 잰 사유(skip) 25 — 클라우드 세션 17 등
`review.reason`	맞음·틀림의 이유 글	357/352	10-13	—	자유 글. 다른 기록 id를 가리킨다
`review.wrong` · `why`	틀린 조각 id · 까닭 넷 중 하나	115 · 101	10-19(p04)	p03 뒤 빗나가면	조건 흐림 94 · 이유 틀림 3 · 바깥 상황 3 · 까닭 모름 1
`review.report`	작업 보고서 경로	98/96	11-02	11-03	다른 저장소 경로 41

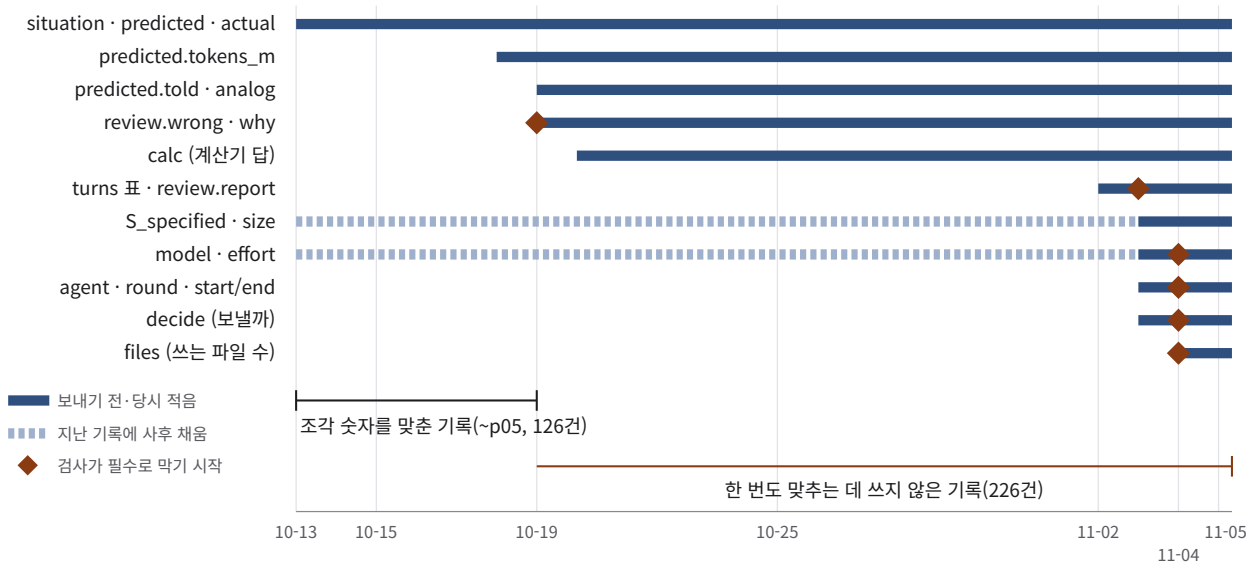


그림 1. 칸별 내력. 계산기의 숫자를 정한 기간(10-13–10-19, 파일 순서로 p05까지)에는 일의 크기를 담는 칸이 하나도 없었다. 크기 칸(`size`)과 '정해진 일' 스위치는 11-03에 결과를 가린 채 작업 설명에서 거꾸로 채운 것이고, 지시서에서 직접 센 파일 수(`files`)는 11-04부터만 있다.

2.3 상황 값: 상태, 스위치, 대 보기

계산기의 입력은 상황 값이다. '상태'란 지시서를 본 사람이면 누구나 같은 답을 낼 사실 질문(예: S_fixed_input "입력이 정해진 작은 파일 묶음이고, 출력이 하나이고, 확인 실행이 없다")이고, 값은 예·아니오·일부·모름 넷 중 하나다. 조각의 조건으로 쓰일 때 상태는 켜고 끄는 '스위치' 구실을 한다. 보낼 때 기록에 상태마다 값을 적는 일을 '대 보기'라 부른다. 지식 파일에는 상태가 36개 있고, 그중 30개가 `known_before = true`, 곧 보내기 전에 알 수 있는 성질이다. 나머지 6개(S_short, S_mid_notes 등)는 끝나 봐야 아는 값이라 계산의 조건으로 쓰면 테스트가 막는다.

적히지 않은 상태의 값은 정해진 순서로 풀린다(`predict.resolve`). 기록에 적힌 값이 먼저이고, 없으면 저장소에서 정해지는 값(S_small_codebase의 `from_project`), 그다음 기본값(S_other_model과 S_many_turn_model은 '아니오'), 마지막이 '모름'이다. 여기에 데이터의 함정 하나가 숨어 있다. 조각의 조건 '!예'(예가 아님)는 코드상 값이 '예'와 다르기만 하면 통과하므로 '모름'도 통과한다(`predict.matches`). 끝난 기록에 지금 조각을 돌려 세어 보면 '모름'으로 '!예'를 통과한 효과 적용이 70회다. 조각별로는 C_turns_no_check_v2가 34회, C_turns_resumed_v2가

23회, C_small_codebase_v2가 9회, C_turns_research_v2가 4회다. 게다가 그 '모름'의 대부분은 판단해 보니 모르는 것이 아니라 그 상태를 묻기 시작하기 전이라 '안 물은' 것이다(p179 보고서는 상태 9개의 '모름' 1,735칸 중 74 %가 그렇다고 셌다, 2026-11-04).

표 5. 계산 조각이 쓰는 상태의 적힌 수와 값 분포(전체 362건). '적힘'에 들지 않은 기록은 풀이 순서를 거쳐 대개 '모름'이 된다.

상태	뜻(줄임)	적힘	값 분포
S_other_model	Opus 5.5 high가 아닌 설정	326	예 92 · 아니오 234 (11-04부터 model·effort에서 계산)
S_fixed_input	정해진 작은 입력, 출력 하나, 확인 없음	320	예 97 · 일부 28 · 아니오 195
S_resumed	앞 회차를 마친 같은 에이전트에 이어 보냄	325	예 82 · 아니오 243
S_web	웹 검색·읽기가 주된 일	324	예 67 · 일부 1 · 아니오 256
S_research	엄격한 양식, 코드 없는 조사	249	예 75 · 일부 7 · 아니오 167
S_fix_existing	기존 것을 고치는 일이 중심	220	예 67 · 일부 13 · 아니오 140
S_no_code	코드를 짓지도 고치지도 않음	189	예 138 · 일부 2 · 아니오 49
S_external_check	외부 시스템 확인이 들어감	352	예 139 · 일부 10 · 아니오 203
S_integration	여러 갈래를 합치는 일	212	예 10 · 아니오 202
S_many_turn_model	턴을 잘게 나눠 쓰는 모델 설정	245	예 11 · 아니오 197 · 모름 37 (기본값 아니오)
S_small_codebase	검사가 몇 초면 끝나는 작은 코드베이스	88	예 55 · 아니오 18 · 모름 15 (나머지는 저장소로 정함)
S_similar_past	비슷한 크기의 지난 일이 있다	338	예 237 · 일부 47 · 아니오 54
S_specified	할 일이 지시서에 다 적혀 있다(정해진 일)	340	예 205 · 일부 35 · 아니오 100 (269건 사후 채움)
S_full_brief (참고)	지시서와 약속 문서가 완전하다	334	예 333 · 일부 1 — 아무것도 가르지 못한다

2.4 알려진 치우침과 함정

표본을 다룰 때 걸려 넘어지기 쉬운 곳을 정리하면 다음과 같다. 대부분 기록 파일의 머리 주석이나 돌아보기 글에 이미 적혀 있지만 한곳에 모여 있지 않았다.

1. **사후에 채운 값.** 상태 여럿(10-16, 10-19, 10-23), S_specified와 `size`(11-03, 269건), `model·effort`(11-03, 261건)가 결과가 이미 있는 기록에 거꾸로 채워졌다. `size`와 S_specified는 결과를 가린 하위 에이전트가 작업 설명 요약만 보고 채웠으므로 결과 누설은 막았지만, 지시서 원문이 아니라 한두 줄 요약에서 센 것이라 수가 빠지고 단위가 섞였다. `files`는 결과 뒤에 채운 것에 `after = true`를 달아 시험에서 기계적으로 빼도록 했다(11건).
2. **파일 순서 ≠ 날짜 순서.** 브랜치를 합칠 때마다 번호를 다시 매겨 붙였다(11-02 p107–p137, 11-03 p145–p167, 11-05 p220–p236 = 원래 p195–p211). 그 결과 파일에서 앞 기록보다 날짜가 이른 기록이 49건이다. 그런데 '세운 뒤의 기록으로만 시험한다'는 원칙(`SEEN_UNTIL`, 5.1절)은 파일 순서로 자른다. 날짜순 분석을 할 때는 반드시 `date`(같은 날이면 `actual.start`)로 다시 정렬해야 한다.
3. **같은 지시서 묶음.** 표본은 독립이 아니다. 11-03의 먼 분야 생성 28건(p148–p167, p171–p178)은 같은 지시서를 범위만 바꾼 것이고, 11-04–05의 exam_grader 풀이지 12건은 한 에이전트의 14–25회차다. 이런 묶음 하나가 날짜 구간 하나를 거의 다 차지하므로, 묶음 단위로 빼 보는 시험(leave-one-cluster-out)이 없으면 결론이 한 묶음에 끌려간다(7.3절의 예).
4. **저장소 쏠림.** 조각의 숫자는 batman 시기의 기록으로 맞췄고, 파일 수 증거는 두 저장소(exam_grader 12, gemmin 7)에서만 나왔다. 저장소는 일의 갈래를 대신 가리키는 강한 변수다(p179 보고서: 분의 퍼짐을 저장소 하나가 η^2 0.42, 상태 하나는 0.30 이하, 2026-11-04).
5. **재는 단위.** 턴은 블록, 토큰은 줄마다 더한 값이다(1.1절). 생각 블록은 0글자로 남아 생각하는 양을 글자로 잴 수 없다. 클라우드 세션으로 보낸 일은 이 세션에 작업자 대화 기록이 없어 턴 길이 표를 못 잰다(skip 17건).
6. **계산기 답의 이질성.** `calc` 칸에는 판과 종류가 적히지 않는다. 그때그때의 조각 판, 스위치만 쓴 계산과 빗댄 기록을 쓴 비유 계산, 잘못 넣은 상황 값(p138–p144의 S_resumed 누락, p139·p142·p169·p170의 S_other_model)이 한

칸에 섞여 있다. 그러므로 '손 예측 vs 계산기' 집계는 '그때 메인이 받은 계산기 답'의 성적이지 지금 계산기의 성적이 아니다.

7. **구간 기준선의 이동.** 표 3처럼 실제 구간의 분포가 기간마다 다르다. 적중률만 보고 나아졌다고 판단하면 기준선의 이동을 실력으로 착각한다.

---

3. 계산기의 구조

3.1 조각과 효과

계산기의 부품을 '조각'이라 부른다. 조각은 지식 파일 `knowledge.toml`의 가설 가운데 '효과'(`effect`) 칸이 붙은 살아 있는 것이다. 이 저장소에서는 규칙도 '검증됨' 상태 없이 성적이 붙은 가설로 다루므로(2026-10-19 결정), 조각 역시 참으로 확정된 사실이 아니라 시험을 계속 받는 주장이다. 효과 하나는 다섯 칸으로 이루어진다. 무엇에(`on`: 턴, 분당 턴, 분, 턴당 맥락(천 토큰), 더하는 분), 어떻게(`op`: 기준값 base, 곱하기 mul, 더하기 add, 비율 scale), 범위(`range` [하한, 상한]), 언제(`when`: 상태 = 값들, '!'를 붙이면 부정), 방향(`dir`: 곱하기가 늘리는지 줄이는지의 주장)이다. 여기에 효과마다 숫자의 이유(`reason`)가, 조각마다 맞출 때 본 마지막 기록(`seen_until`)과 맞춘 근거(`fit`)가 붙는다. 지금 살아 있는 조각은 숫자를 맞추는 계산 조각 16개(효과 17개)와 숫자가 없는 비율 조각 1개이며, 더하기 효과를 가진 살아 있는 조각은 없다.

3.2 조합: 곱하기와 로그 폭

계산은 네 수량을 각각 '기준값 × 곱하기들'로 만든 뒤 엮는다. 턴 T, 분당 턴 R, 턴당 맥락 C에 대해, 상황 값에 맞는 기준값 조각 하나 B와 맞는 곱하기 조각 모두 m_k를 곱하고, 분은 턴을 분당 턴으로 나누어 더하는 분을 보탠다. 단, 분에 직접 걸리는 기준값(다른 모델의 정해진 입력 풀이, C_min_other_fixed)이 맞으면 그것이 분을 정한다.

$$T = B_{\mathrm{T}} \cdot \prod m_k, \quad R = B_{\mathrm{R}} \cdot \prod m_k, \quad C = B_{\mathrm{C}} \cdot \prod m_k \tag{2}$$

$$\text{분} = T / R + \Sigma\, add_j \quad (\text{분 기준값이 맞으면 분} = B_{\mathrm{M}} \cdot \prod m_k), \quad \text{토큰(M)} = T \cdot C / 1000 \tag{3}$$

각 항은 점이 아니라 범위이므로 범위를 합치는 규칙이 필요하다. 계산기는 범위 $[l, h]$ 하나를 로그 척도의 중심 $c = \tfrac{1}{2}\ln(lh)$와 반폭 $w = \tfrac{1}{2}\ln(h/l)$로 보고, 곱하는 항들을 서로 독립인 불확실성으로 가정해 중심은 더하고 반폭은 제곱합의 제곱근으로 합친다(`predict.combine`). 나눗셈은 역수 범위 $[1/h, 1/l]$를 곱하는 것으로 처리한다.

$$[L, H] = \exp\left(\Sigma c_k \mp \sqrt{(\Sigma w_k^{\,2})} \right) \tag{4}$$

마지막으로 합친 범위의 폭이 분야 상한 `max_ratio = 2`를 넘으면 기하 중심 둘레로 자른다. 사용자에게 한 구간을 말해야 하므로 범위가 끝없이 넓어지는 것을 막는 장치다.

$$H/L > 2 \text{ 이면 } [L, H] \leftarrow \left[\ \sqrt{(LH)}/\sqrt{2}\ ,\ \sqrt{(LH)}\cdot\sqrt{2}\ \right] \tag{5}$$

자르기는 분과 토큰에서는 맨 마지막에 한 번 일어나고, 턴은 보여 줄 때만 잘린다. 즉 분은 자르기 전 턴 범위와 분당 턴 범위를 (4)로 합친 뒤 더하는 분을 하한·상한에 그대로 보태고 나서 (5)를 적용한다. 이 자르기는 예외가 아니라 거의 늘 일어난다. 끝난 기록 중 계산기가 분을 낸 285건 가운데 276건이 잘렸으므로, 사용자가 받는 분 범위는 사실상 언제나 정확히 2배 폭이고 그 폭은 확신의 정도에 대해 아무것도 말하지 않는다(p179 보고서 A3이 같은 점을 짚었다).

3.3 기준값 고르기, '모름', 그리고 비유 경로

한 수량에 맞는 기준값이 여럿이면 조건(`when`)의 개수가 가장 많은, 즉 가장 좁은 것 하나를 고르고, 같으면 파일에서 앞선 것을 쓴다. 맞는 기준값이 하나도 없으면 계산기는 숫자를 지어내지 않고 '모름'을 낸다. 지금 턴·분당 턴·맥락의 기준값은 모두 S_other_model = 아니오(기본 설정)에만 걸려 있고, 기본이 아닌 설정에는 정해진 입력 풀이의 분

기준값 하나만 있다. 그래서 끝난 352건 중 67건이 '모름'이며, 67건 모두 기본이 아닌 설정이면서 정해진 입력이 아닌 일이다(Haiku 48 · Opus xhigh 7 · Opus medium 5 · Fable 4 · Sonnet max 3). 최근일수록 이 몫이 커져 11-04 이후 55건 중 22건이 '모름'이다.

보내기 전에 적은 빗댄 기록(`predicted.analog`)이 있고 그 기록이 끝나 있으면 계산기는 다른 길, 곧 '비유 경로'를 따로 계산한다. 이때 턴의 기준값은 비유 조각 C_turns_analog로, 빗댄 기록의 실측 턴에 ×0.724–1.37을 곱한 범위다. 빗댄 기록의 실측이 이미 그 일의 모양을 담고 있다고 보아 곱하기 조각은 얹지 않는데, 유일한 예외가 비율 조각 C_turns_files_scale이다. 두 기록 모두에 `files` 칸이 있으면 write·edit 각각의 '이번 수 ÷ 빗댄 기록의 수' 중 큰 쪽을 배수로 곱한다(`predict.count_ratio`).

$$T_{\text{비유}} = T_{\text{빗댄}} \times [0.724,\ 1.37] \times \max\left(w/w',\ e/e' \right) \tag{6}$$

여기서 w, e는 이번 기록의 write·edit 수, '는 빗댄 기록의 것이고, 비는 두 쪽 수가 모두 0보다 큰 칸에서만 낸다.

분당 턴과 맥락은 비유 경로에서도 기준값 조각 그대로다(빗댄 기록에 실측이 있으면 그 수량도 비유 기준을 쓸 수 있게 짜여 있지만, 지금 비유 효과는 턴에만 있다). 두 경로의 답은 따로 기록·채점된다. 보낼 때 `calc`에 적는 것은 원칙상 스위치 경로(`predict <id> --calc`)다.

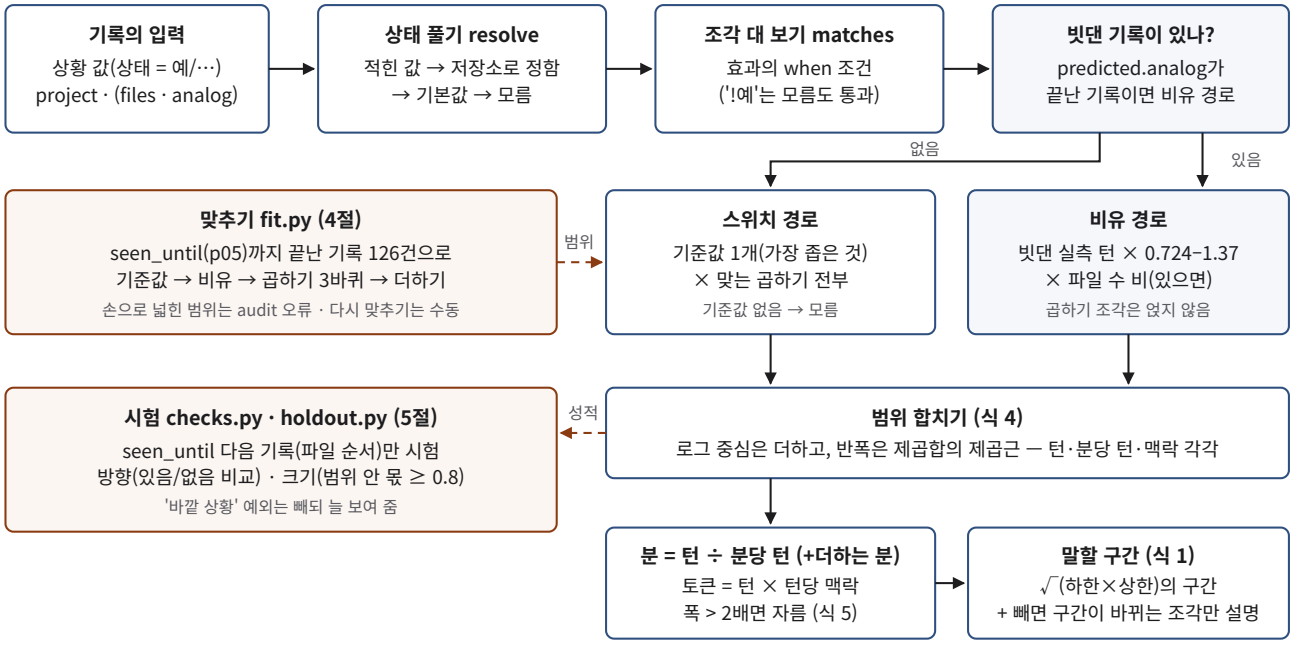


그림 2. 계산기의 흐름. 위 줄과 오른쪽 아래가 보낼 때 한 번 도는 계산(`predict.py`)이고, 왼쪽 두 상자는 그 계산이 쓰는 숫자를 정하는 맞추기와 그 숫자를 평가하는 시험이다. 두 경로의 답은 따로 기록되고 따로 채점된다.

3.4 살아 있는 조각 일람

표 6은 지금 계산에 들어가는 조각 전부다. '맞춤'은 숫자를 정할 때 그 효과가 걸린 기록 중 범위 안에 든 수이고(`fit final`), '시험 크기'는 seen_until 다음의 끝난 기록에서 그 효과가 걸린 기록 중 범위 안에 든 수다(`proviso check`, 바깥 상황 예외 p06·p07·p130을 뺌). 맞춤 쪽은 대부분 잘 담고, 시험 쪽은 대부분 담지 못한다. 이 대비가 이 글 전체의 출발점이다.

표 6. 살아 있는 조각(계산 조각 16 + 비율 조각 1). 조건의 '≠예'는 '!예'. 방향은 시험 기록에서 조건 있음·없음을 비교한 결과이고 기준값은 방향이 없다.

조각	무엇에·어떻게	조건(when)	범위	맞춤	시험 크기	방향
C_turns_base	턴 기준	other_model=아니오	216–426	26/43	24/151	—
C_turns_fixed_input	턴 ×	fixed_input=예	0.0362–0.066	4/8	1/63	지지
C_turns_research_v2	턴 ×	research=예, fixed_input≠예	0.273–0.455	9/10	3/7	지지
C_turns_no_check_v2	턴 ×	external_check=아니오, research·fixed_input·resumed·fix_existing≠예, no_code=예	0.158–0.307	10/13	2/6	지지
C_turns_resumed_v2	턴 ×	resumed=예, integration≠예	0.171–0.333	9/19	8/46	지지
C_fix_existing_v2	턴 × / 맥락 ×	fix_existing=예	0.487–0.97 / 0.413–0.742	8/9 · 9/9	12/43 · 22/43	지지(둘)
C_small_codebase_v2	턴 ×	small_codebase=예, no_code≠예	0.313–0.624	4/5	3/13	지지
C_turns_analog	턴 비유 기준	similar_past=예/일부	×0.724–1.37	6/7	시험 안 함*	—
C_rate_base	분당 턴 기준	other_model=아니오	3.33–6.64	66/94	51/151	—
C_rate_web	분당 턴 ×	web=예	1.44–2.38	8/8	2/5	지지
C_min_other_fixed	분 기준	other_model=예, fixed_input=예	13.6–25.3	6/8	0/8	—
C_min_light_model	분 ×	other_model=예, fixed_input=예, many_turn_model=아니오	0.339–0.626	6/8	0/7	지지
C_ctx_base	맥락 기준(천)	other_model=아니오	227–453	41/55	49/150	—
C_ctx_web	맥락 ×	web=예	0.263–0.504	8/8	1/5	지지
C_ctx_fixed_input	맥락 ×	fixed_input=예	0.187–0.297	8/8	5/63	지지
C_ctx_resumed	맥락 ×	resumed=예	1.09–2.13	18/23	24/46	지지
C_turns_files_scale	턴 ×비율(비유 경로만)	similar_past=예/일부, 두 기록에 files	파일 수 비	0건	0/4	**반박 0/4**

* 조각별 성적(`holdout.piece_scores`)은 비유 효과를 건너뛴다(`if e.analog: continue`). 그래서 비유 경로가 136건에 쓰였는데도 C_turns_analog의 성적표는 늘 '시험 전'이다(부록 A).

3.5 한 기록의 계산: p224

추상적인 식을 한 건에 대 보자. p224는 같은 에이전트가 이어 쓰는 풀이지 시리즈의 17회차(풀이지 세 장, 기본 설정)이고, 상태는 S_resumed = 예, S_fixed_input = 아니오, S_specified = 예 등이며 빗댄 기록으로 p220(풀이지 한 장, 실측 15턴)이 적혀 있었다. 스위치 경로에서는 C_turns_base(216–426)에 C_turns_resumed_v2(×0.171–0.333)가 곱해져 턴이 로그 중심 72.4, 합친 반폭 0.476으로 45–116턴이 되고, C_rate_base(분당 3.33–6.64턴)로 나누면 8.5–27.7분이 되며, 이것이 2배를 넘어 10.9–21.8분으로 잘린다. 가운데 값 15.4분이므로 '다른 일'이다. 실측은 3.1분·25턴으로 '잠깐'이었으니 빗나갔고, 갈래는 턴(양)이 틀린 쪽이다. 비유 경로에서는 p220의 15턴 × [0.724, 1.37] × 파일 비 3.0(write 3 ÷ 1)으로 32.6–61.7턴, 분 6.7–13.5, 가운데 9.5분 '잠깐'이 되어 구간은 맞았지만 턴 범위는 실측 25를 담지 못했다. 손 예측은 5–8분·25–40턴으로 둘 다 맞았다. 한 건이지만 세 가지가 함께 보인다. 스위치 경로는 '이어 보냄'이라는 스위치 하나로 큰 구현 기준값을 깎을 뿐 풀이지 한 장의 크기를 모르고, 비유 경로는 크기를 빗댄 기록에서 가져오지만 파일 비례가 지나치며, 손은 바로 앞 회차들의 시간을 직접 본다.

3.6 설명에 남는 조각

계산기는 숫자만 내지 않고 '근거 → 조각 → 조합 → 숫자'의 설명을 붙인다. 다만 쓴 조각을 모두 늘어놓지 않고, 그 조각 하나를 뺐을 때 말할 구간이나 토큰 등급이 바뀌는 조각만 설명에 남긴다(`Prediction.decisive`). 위 p224의 스위치 경로에서 C_ctx_resumed는 계산에 쓰였지만 빼도 구간·등급이 같아 설명에서 빠진다. 이 규칙은 '빼면 결론이 바뀌는 조각만 붙인다'는 저장소 전체의 원칙을 옮긴 것이다.

4. 맞추기(fit)

4.1 창 방법

조각의 모양, 즉 무엇에·어떻게·언제·방향·이유는 사람(실제로는 AI)이 손으로 쓰고, 범위 숫자만 `fit.py`가 기록에서 기계적으로 정한다. 숫자를 정하는 핵심 함수는 '창'(`fit.window`)이다. 조각이 걸린 기록마다 값을 하나씩 만든 뒤, 값들을 정렬하고 상한/하한 ≤ 2인 창을 밀어 가며 가장 많은 값을 담는 창을 찾는다. 담는 수가 같으면 창의 로그 중심이 값들의 가운데 값에 가까운 쪽을 고르고, 고른 창을 그 안에 든 값의 최솟값–최댓값으로 줄인 뒤 유효숫자 셋으로 바깥쪽 반올림한다.

$$\text{범위} = [\min S,\ \max S],\quad S = \operatorname{argmax}_{\text{창}} \left|\{\, v : v_i \le v \le 2v_i \,\}\right| \tag{7}$$

값이 하나뿐이면 그 값 둘레의 가장 넓은 창 $[v/\sqrt{2},\ v\sqrt{2}]$를 주되 audit이 '근거 1건'으로 경고하고(한 건이라도 기본 논리가 틀린 게 보이면 고친다는 2026-10-19 결정), 값이 여럿인데 어느 창도 둘을 담지 못하면 조각을 세우지 않는다(크기가 흩어진 조건은 더 쪼갤 조건이라는 뜻). 이 범위는 확률 구간이 아니라 '가장 붐비는 2배 창'이라는 점이 중요하다. 맞춘 기록에서조차 기준 턴은 43건 중 26건만 담는다. 크기 시험의 잣대(범위 안 몫 ≥ 0.8, 5.2절)는 이 만드는 방식과 맞지 않아서, 맞춘 기록에서도 대부분의 조각이 그 잣대를 넘지 못한다.

4.2 맞추는 순서

맞추기는 네 단계로 진행한다. 첫째, 기준값은 그 기준값이 고른 기록 중 **곱하기 조각이 하나도 걸리지 않은** 기록의 실측으로 맞춘다. 그래서 C_turns_base는 실질적으로 '다른 조건이 하나도 없는, 확인 고리가 있는 첫 회차 구현'의 턴이다. 둘째, 비유 기준값은 빗댄 기록이 있는 기록의 '실측 ÷ 빗댄 기록의 실측'으로 맞추는데, 맞출 때만 돌아보기 글에서 사후에 읽은 빗댄 기록 다섯(`checks.PAST_ANALOG`)도 쓴다. 셋째, 곱하기는 걸린 기록의 값을 아래처럼 만들어 창에 넣되, 곱하기끼리 서로의 가운데 값을 쓰므로 세 바퀴 돌려 맞춘다. 맞춘 범위가 주장한 방향과 어긋나면(줄인다고 했는데 1을 넘나들면) 방향이 데이터와 맞지 않는 조각으로 표시된다.

$$v = \text{실측} \,/\, \left(\text{중심(기준값)} \times \prod_{\text{다른 곱하기}} \text{중심} \right) \tag{8}$$

넷째, 더하기는 '실측 분 − 이 조각을 뺀 계산의 가운데 분'으로 맞춘다(지금 살아 있는 더하기는 없다). 비율 조각은 범위 숫자가 없으므로 맞추지 않는다.

4.3 seen_until과 수동 다시 맞추기

'seen_until'은 조각을 세우거나 숫자를 맞출 때 본 마지막 기록의 id다. 계산 조각 16개는 모두 seen_until = p05(2026-10-19, 파일 순서로 끝난 기록 126건)이고, `fit final`은 이 값이 하나여야만 돈다. 비율 조각은 숫자가 없어 그 구역 밖에 두고 seen_until = p186을 따로 단다. 2026-11-03 사용자는 숫자를 자동으로 다시 맞추지 않기로 결정했다. 정리할 때 사람이 seen_until을 옮기고 `fit final`로 바뀐 범위를 본 뒤 `--write`로 쓰며, 지식 파일의 범위를 손으로 고치면 audit 오류가 난다. 다시 맞출 때 방향이 뒤집히거나 값이 흩어져 범위를 못 잡는 조각은 그대로 받아들이지 말고 이유가 틀린 신호로 따지라는 단서가 붙어 있다. 그 결과 p05 다음의 226건은 한 번도 숫자를 맞추는 데 쓰이지 않았다.

4.4 오늘의 시험 삼아 다시 맞추기

파일을 쓰지 않고 끝난 352건 전부로 다시 맞춰 보면 표 7과 같다 [사후]. 기준 턴은 내려가고(216–426 → 163–325), 정해진 입력의 턴 배수는 세 배 넘게 오르며(×0.036–0.066 → ×0.117–0.234), C_small_codebase_v2는 1을 넘나들어 방향이 뒤집힌다. 가장 큰 변화인 C_min_light_model(×0.34–0.63 → ×0.086–0.16)은 속도에 대한 새 사실이 아니라 묶음의 문제다. 다시 맞출 때 걸린 15건이 Fable xhigh 8건과 Opus xhigh 7건(1–2분짜리 풀이지)인데, S_many_turn_model의 기본값이 '아니오'라서 '턴을 잘게 쓰지 않는 모델'에 성격이 전혀 다른 두 설정이 같이 들어간 것이다.

표 7. 지금 범위(p05까지 126건) vs 끝난 352건 전부로 다시 맞춘 범위. 쓰지 않고 보기만 했다(`refit_dry.py`). 가운데 비 = 새 중심 ÷ 지금 중심.

조각	효과	지금 범위 (설명/걸림)	다시 맞춘 범위 (설명/걸림)	가운데 비	표시
C_turns_base	턴 기준	216–426 (26/43)	163–325 (35/60)	×0.76	
C_turns_fixed_input	턴 ×	0.0362–0.066 (4/8)	0.117–0.234 (39/71)	×3.39	
C_turns_research_v2	턴 ×	0.273–0.455 (9/10)	0.36–0.7 (15/19)	×1.42	
C_turns_no_check_v2	턴 ×	0.158–0.307 (10/13)	0.208–0.405 (12/19)	×1.32	
C_turns_resumed_v2	턴 ×	0.171–0.333 (9/19)	0.195–0.386 (25/65)	×1.15	
C_fix_existing_v2	턴 × · 맥락 ×	0.487–0.97 · 0.413–0.742	0.49–0.967 · 0.416–0.805	×1.00 · ×1.05	
C_small_codebase_v2	턴 ×	0.313–0.624 (4/5)	0.553–1.06 (10/20)	×1.73	**방향 뒤집힘**
C_turns_analog	비유 기준	0.724–1.37 (6/7)	0.703–1.4 (80/134)	×1.00	
C_rate_base	분당 턴 기준	3.33–6.64 (66/94)	4.04–8.07 (125/241)	×1.21	
C_rate_web	분당 턴 ×	1.44–2.38 (8/8)	1.02–1.96 (12/15)	×0.76	
C_min_other_fixed	분 기준	13.6–25.3 (6/8)	13.6–25.3 (6/9)	×1.00	
C_min_light_model	분 ×	0.339–0.626 (6/8)	0.0862–0.162 (7/15)	×0.26	설정 둘이 섞임
C_ctx_base	맥락 기준	227–453 (41/55)	209–418 (52/84)	×0.92	
C_ctx_web · C_ctx_fixed_input · C_ctx_resumed	맥락 ×	0.263–0.504 · 0.187–0.297 · 1.09–2.13	0.285–0.56 · 0.248–0.495 · 1.17–2.31	×1.10 · ×1.49 · ×1.08	

5. 시험과 장치

5.1 세운 뒤의 기록으로만 시험한다

이 분야의 시험 원칙은 하나로 요약된다. 가설은 그것을 세운 뒤에 나온 기록으로만 시험하고, 세울 때 본 기록은 '맞춰 본 것'으로 따로 보여 준다. 사후 설명, 곧 결과를 보고 그 결과에 맞는 이유를 지어 붙이는 일을 막기 위해서다. 손 규칙과 일반 가설은 `checks.SEEN_UNTIL`의 경계를, 계산 조각은 조각마다의 `seen_until`을 쓴다. 경계는 파일 순서이므로 2.4절의 번호 다시 매기기와 맞물린다. 계산 조각의 시험 기록은 p05 다음의 끝난 기록 223건(바깥 상황 예외를 뺀 수)이다.

'바깥 상황 예외'란 돌아보기에서 까닭을 "바깥 상황"(보내기 전에 알 수 없던 드문 일)으로 가른 기록이다. 이런 기록은 계산 조각의 방향·크기 시험에서 빠지지만 숨지는 않아서, 성적표와 audit 경고에 '예외로 뺀 기록'으로 늘 나온다. 지금 p06·p07(연결이 여러 번 끊긴 날의 웹 조사)과 p130 세 건이다. 같은 종류가 되풀이되면 예외가 아니라 빠진 조건이므로 쪼개야 한다는 단서가 붙어 있다. 여기서 '쪼갠다'는 것은 숫자를 넓히는 대신 그 차이를 만드는 상태를 조건에 더해 조각을 둘로 나누고, 옛 조각을 이전 버전으로 남기는 일이다.

5.2 방향 시험과 크기 시험

조각 하나의 성적은 둘로 나뉜다(`holdout.piece_scores`). '방향 시험'은 곱하기 조각이 주장한 방향(늘린다/줄인다)이 맞는지를 본다. 시험 기록마다 '실측 ÷ 그 조각을 뺀 계산의 가운데'를 구하고, 같은 기준값을 고른 기록끼리 조건이 있는 쪽과 없는 쪽의 가운데 값을 견준 뒤, 묶음마다 min(있음 수, 없음 수)로 무게를 두어 로그 차이를 평균한다. 그 부호가 주장과 같으면 '지지'다. '크기 시험'은 조건이 걸린 시험 기록 중 그 비율이 조각 범위 안에 든 몫이며, 0.8 이상이면 지지다(`NEED = 0.8`). 방향이 틀리면 이유(논리)가 깨진 것으로 보아 audit 오류가 되고, 크기만 틀리면 조건이 흐린 것으로 보아 쪼개기 후보가 된다. 주의할 점은 방향 시험이 효과마다 비교 한 번을 '1건'으로 센다는 것이다. 성적표의 '지지 1/1'은 기록 한 건이 아니라 가운데 값 비교 한 번의 결과다.

5.3 홀드아웃

조각 체계를 처음 세운 2026-10-19 실험은 시간 순서 홀드아웃으로 남아 있다(`holdout.py`). 끝난 기록을 d38b에서 앞 64건과 뒤로 나누고, 뒤 절반의 끝은 p05로 고정했다(2026-10-22, 새 기록이 섞여 성적이 흔들려서). 앞 절반으로만 맞춘 두 세트(쪼개기 전 front_v0, 쪼갠 뒤 front_split, `holdout.toml`)를 뒤 62건으로 시험하면 말할 구간 적중이 쪼개기 전 24/62(39 %), 쪼갠 뒤 31/62(50 %), 당시 손 예측 39/62(63 %)다 [시험]. 이 홀드아웃은 10-16–10-19 나흘치의 batman 중심 기록이라 '같은 시기 안에서 옮겨 가나'만 재고, 새 저장소의 일로 옮겨 가는지는 재지 않는다.

5.4 audit과 장치

`python -m proviso audit agent_usage`는 그래프와 성적에서 기계로 볼 수 있는 문제를 오류와 경고로 낸다. 계산 조각에 대해서는 시험 기록에서 방향이 틀린 조각, 지식 파일의 범위가 기록에서 맞춘 값과 다른 조각(손으로 넣은 숫자), 설명하는 기록이 없는 효과, 효과 수가 설명한 기록에 비해 많은 경우, 쪼갰는데 '왜 이 조건이 차이를 만드나'(`why_condition`)가 없는 경우가 오류이고, 근거 1건, 쪼개지 않은 개정, 시험 수단 없음, 예외로 뺀 기록이 경고다. 2026-11-05 현재 오류 1건(C_turns_files_scale 방향 0/4)과 경고 24건(쪼개지 않은 개정 14 · 시험 수단 없음 8 · 분야 점검 2)이며, 이 오류 때문에 테스트 149개 중 `test_audit_has_no_errors_now` 하나가 실패한다. 표 8은 `tests/test_computable.py`가 지키는 장치를 정리한 것이다.

표 8. 계산되는 논리를 지키는 장치(코드·테스트가 막음). 글자는 테스트 이름의 머리 글자.

장치	내용	막는 것
A 방향	조건 조각은 방향 주장이 있어야 하고 범위와 맞아야 한다(1을 넘나들면 방향 없음). 시험 기록에서 방향이 틀리면 audit 오류	논리 없는 숫자 조정
B 이유	효과마다 숫자의 이유, 쪼갠 조각에는 '왜 이 조건이 차이를 만드나'	이유 없는 숫자
C 보내기 전	조건에는 `known_before = true`인 상태만	결과 누설
D 한 건	값 하나면 가장 넓은 창 + 경고, 흩어진 여럿이면 세우지 않음	흐린 조건을 숫자로 덮기
E 회귀 금지	쪼갠 뒤 맞춰 본 기록에서 새로 틀리는 기록은 `REGRESSION_OK`에 이유가 있어야 함(d15b, d20·d25·d57)	쪼개기로 다른 곳을 망가뜨리기
F 효과 수 상한	효과 수 × 4 ≤ 맞춘 기록에서 말할 구간을 맞힌 수(지금 17 × 4 = 68 ≤ 89)	외워서 맞추기
G 폭·손 숫자	범위 폭 ≤ 2배, 범위는 `fit`이 낸 값과 같아야 함	손으로 넓히기
H 맨 윗줄	성적표 맨 윗줄은 말할 구간	사용자가 받지 않는 숫자로 자랑하기
그 밖	조각 이름·문장에 저장소·앱 이름 금지, 비율 조각은 숫자 없이 수 칸 필요, 비유 기준에만 얹힘	분야 간 재사용 방해, 이중 계산

---

6. 주변 모델

6.1 손 예측 규칙

손 예측은 지식 파일의 규칙(R_*, 44개 중 23개가 살아 있음)을 참고한다. 규칙은 "같은 종류의 직전 실측이 있으면 그 턴·분 ×1"(R_turns_analogy_v2), "분 = 턴 ÷ 분당 턴, 구현·문서는 분당 2.5–6.5턴"(R_time_turns_v2) 같은 문장과 숫자이고, 그 규칙을 `predicted.rules`에 적은 예측이 맞았는지로 '쓴 예측' 성적을 받는다. 예컨대 R_turns_analogy_v2는 턴 107/166, 분 113/206, R_time_turns_v2는 분 102/200이다. 이 성적은 규칙 여럿을 섞은 예측의 성적이라 규칙 하나의 탓을 가리지 못하므로, 빗나간 기록에는 돌아보기에서 틀린 조각을 `review.wrong`으로 짚게 한다. 손 예측이 계산기를 줄곧 이긴 까닭에 대해 p179 보고서(2026-11-04)는 손이 지시서 본문, 즉 만들 것의 수와 무게와 시리즈의 몇 번째인지를 보고 빗댄 기록에서 출발해 고친다는 점을 들었고, 그 '고치는 몫'을 10 %p쯤으로 셌다 [사후].

6.2 빗댄 기록 규칙

손이 감으로 고르던 빗댄 기록을 누구나 같은 기록을 고르는 기계 규칙으로 바꾸려는 시도가 `analog.py`다. 정해진 일(S_specified = 예)이면 보낼 때 이미 끝나 있던 기록 중에서 반드시 같은 답(S_specified, S_resumed, S_web, S_fixed_input, S_cloud_session)과 같은 모델·effort를 요구하고, 같은 저장소, 그다음 부드러운 상태 여섯의 일치 수로 순위를 매기며, 맨 앞이 동점이면 그 묶음 실제 분의 가운데 값을 쓴다. v1은 2026-11-03 시험에서 규칙 84 % 대 AI가 고른 기록 94 %로 실패했다. v2는 같은 에이전트의 앞 회차를 맨 앞 기준으로 두고, 동점끼리 2배 넘게 벌어지면 답하지 않고 손으로 넘기며, 범위를 ×0.759–1.51로 맞췄다. 미리 정한 판정 조건(새 시험 기록 30건, 규칙이 AI보다 5 %p 넘게 낮지 않을 것)이 이제 충족되었다. 시험 기록 32건에서 규칙은 22건(69 %), AI가 고른 빗댄 기록은 31건(97 %), 손 예측은 32건(100 %)을 맞혔으므로, 미리 정한 기준으로 **v2는 실패**다 [시험]. 분야 문서(CLAUDE.md)는 아직 v2를 '시험 전'으로 적고 있다.

6.3 일의 크기 모델

`size.py`는 정해진 일의 턴을 '기본 몫 + Σ(센 수 × 하나마다 몫)'으로 계산하는 모델이다. 센 수는 `size` 칸(읽을 것·만들 것·쓸 덩어리·고칠 곳·확인)이고, p99까지의 기록으로 몫을 맞췄다. 그러나 그 칸은 2.4절에서 말했듯 작업 설명 요약에서 사후에 센 것이고 단위가 섞였다. 맞추는 기록 88건 안에서도 몫마다 범위 안에 든 기록이 4–18 %뿐이었고, 다섯 수의 합과 턴의 순위 상관은 +0.21, 읽을 것은 −0.49였다(2026-11-03 첫 시험). 지금 시험 기록은 65건으로 늘었고, 나눈 모델 1/65(모름 40), 나누지 않은 모델 15/65, 스위치만 계산기 3/65, 빗댄 기록에서 출발 43/56, 손 예측 56/65다 [시험]. 이 실패가 '세는 것' 자체의 실패인지 '사후에 요약에서 센 것'의 실패인지는 이 데이터로 가를 수 없다. 그래서 다음 걸음으로 실제 지시서에서 보내기 전에 세는 `files`가 생겼다.

6.4 파일 수 비율 조각

비율 조각 C_turns_files_scale은 2026-11-04 사용자 결정으로 세웠다. 같은 모양의 일이면 쓰는 파일이 n배일 때 턴도 n배라는 주장이고, 비유 경로에서만 걸린다(식 6). 전역 '파일 하나 = 몇 턴'을 세울 수 없다는 관찰(파일 하나당 분이 0.12분부터 16.7분까지 100배, 2026-11-04 확인)에서, 절대량이 아니라 같은 모양의 빗댄 기록에 대한 비로만 쓰기로 한 것이다. 미리 정한 시험은 걸린 기록 15건이 쌓이면 판정하고, 방향은 2건부터 본다. 지금까지 걸린 4건은 표 9와 같고, 네 건 모두에서 비율을 곱한 쪽이 곱하지 않은 쪽보다 실측에서 멀었다 [시험].

표 9. 비율 조각의 시험 기록 4건(턴). '제곱근'은 비율의 제곱근을 곱한 값으로, 이번에 데이터를 보고 계산한 것이라 판정이 아니다 [사후].

기록	빗댄 기록	이번 files	빗댄 files	비	빗댄 실측	비례 예측	제곱근	실측
p189 (gemmin)	p185	edit 3	edit 5 (`after`)	0.6	53	31.8	41.1	61
p203 (gemmin, 조사 종합)	p196	write 1, edit 2	write 5, edit 13	0.2	93	18.6	41.6	91
p214 (gemmin)	p193	write 1, edit 1	write 12	0.083	187	15.6	54.0	63
p224 (exam_grader)	p220	write 3, edit 2	write 1, edit 2	3.0	15	45.0	26.0	25

네 건의 꼴은 한결같다. 실측은 빗댄 기록에서 비율 쪽으로 움직이기는 하지만(p214 187 → 63, p224 15 → 25) 비례보다 훨씬 덜 움직이고, 두 건(p189·p203)은 거의 움직이지 않는다. 이것은 지시서 읽기·둘러보기·확인·보고처럼 파일 수와 무관한 고정 몫이 있다는 가설과도, 비율이 약하게(지수 1 미만으로) 걸린다는 가설과도 들어맞는다 [짐작]. 제곱근은 넷 모두에서 비례보다 가깝지만 '곱하지 않음'보다 가까운 것은 둘뿐이고, 4건을 보고 고른 모양이라 이것을 근거로 삼으면 사후 맞춤이 된다. 덧붙여 코드의 빈틈 하나가 있다. 시험 기록을 고를 때 `holdout._scale_scores`는 이번 기록의 수가 결과 뒤에 채워진 것(`after`)이면 빼지만 빗댄 기록 쪽의 `after`는 보지 않는다. p189의 빗댄 기록 p185는 `after = true`라서, 엄밀히는 p189가 시험에서 빠져야 할 수도 있다.

6.5 보낼까 판단

보낼까 판단(`decide.py`)은 계산기와 다른 질문, 곧 하위 에이전트에게 맡길지 메인이 직접 할지를 같은 단위의 토큰으로 견준다. 직접 하면 일의 토큰에 더해 메인 맥락이 커져 그 뒤의 모든 응답에서 다시 읽히는 비용이 들고(직접 = $T\times(M+A/2) + A\times L$), 보내면 작업자의 토큰과 지시·기록에 드는 메인 줄과 남는 보고의 비용이 든다(보냄 = $S + K\times M + R\times L$). T·S는 손 예측(없으면 계산기)의 턴·토큰, M은 지금 메인 맥락, L은 남은 대화 줄이다. 판단 기록 66건 중 55건에서 고른 쪽이 더 쌌다 시험. 다만 보낸 기록 69건이 모두 '보냄'이고 직접 한 기록은 1건(m001, 실제 22.4M 대 다른 쪽 추정 9.4M으로 직접이 비쌌음)이라, 이 시험은 사실상 한쪽만 본다. 고르지 않은 쪽은 해 보지 않은 추정이라는 한계도 있다.

---

7. 현재 성적과 오늘 확인한 것

7.1 성적표

표 10. 말할 구간 적중. 같은 행 안에서만 견준다(행마다 기록 묶음이 다르다).

비교	기록	계산기	손 예측	출처·성격
보낼 때 둘 다 적은 끝난 기록	161	64 (40 %)	102 (63 %)	dispatches. 둘 다 맞음 47 · 손만 55 · 계산기만 17 · 둘 다 틀림 42. 계산기 쪽은 판·종류가 섞임
지금 조각, p05까지(숫자를 맞춘 기록)	126	89 (71 %)	85 (67 %)	맞춰 봄 계산기 모름 6
지금 조각, p05 뒤(맞출 때 안 본 기록)	226	53 (23 %)	148 (65 %)	시험 계산기 모름 61, 답한 것 중 53/165
홀드아웃 뒤 절반(쪼갠 뒤 세트)	62	31 (50 %)	39 (63 %)	시험 10-16–10-19
말한 구간이 있는 끝난 기록 전체(손)	228	—	149 (65 %)	v2(긴 쪽)와 v3(가운데) 시기가 섞임
R_say_numbers_v3 시험 기록(손)	66	—	60 (91 %)	시험 p145 다음, v3로 말한 것만

표 10의 셋째·둘째 행이 계산기의 실력을 가장 정직하게 보여 준다. 숫자를 맞춘 기록에서는 손과 비슷하거나 낫지만(71 % 대 67 %), 맞출 때 안 본 기록에서는 23 %로 떨어지고 그 넷 중 하나는 답 자체를 못 낸다. p179 보고서(2026-11-04, 끝난 289건 기준)는 같은 구분으로 71 % 대 27 %를 셌고, 곱하기 조각을 많이 쌓을수록 턴 빗나감이 커진다는 점도 짚었다. 지금 다시 세면 턴의 빗나감 배수 가운데 값이 곱하기 0개 60건 ×1.41, 1개 155건 ×2.01, 2개 34건 ×2.84이고, 4개가 쌓인 2건은 ×44.7이다(3개 5건은 ×1.68로 단조롭지 않다). 조각별 시험(표 6)에서도 방향은 곱하기 효과가 모두 지지인데 크기는 모두 반박이다. 곧 계산기의 '이유'는 대체로 맞는데 '크기'가 다른 시기의 일로 옮겨 가지 못한다.

7.2 날짜로 나눈 다시 맞추기 시험

오늘 메인 세션은 "숫자를 새 기록으로 다시 맞추면 나아지는가"를 날짜로 나누어 시험했다(표 11, 다시 돌려 같은 값을 확인했다). 11-02까지 263건으로 맞추고 11-03–05의 89건으로 시험하면, 스위치만 쓰는 계산의 적중이 10/89(11 %)에서 50/89(56 %)로 오른다. 그러나 새로 맞은 41건 중 28건이 같은 지시서 묶음(먼 분야 생성)이고, 그 묶음을 빼면 10/61(16 %)에서 22/61(36 %)으로, 같은 61건에서 손은 54/61(89 %)이다. 또 다시 맞춘 계산기는 답한 64건 중 46건을 '다른 일'이라고 말하는데, 이 시험 기록에서 늘 '다른 일'이라고만 말해도 46/89(52 %)를 맞힌다. 경계를 하루 옮겨 11-03까지 297건으로 맞추고 11-04–05의 55건으로 시험하면 16 %에서 16 %로 전혀 오르지 않으며, 같은 기록에서 손은 50/55(91 %)이고 늘 '잠깐'은 39/55(71 %)다. 비유 경로는 두 시험 모두 조금 나빠진다(79 % → 76 %, 83 % → 77 %).

표 11. 날짜로 나눈 다시 맞추기(`refit_test.py`·`refit_why.py`·`refit_excl.py`). 모름은 틀림으로 센다.

나눔	시험	스위치: 지금 → 다시	비유: 지금 → 다시	손	모름	기준선
≤11-02 맞춤 / 11-03–05 시험	89	10 → 50 (11 % → 56 %)	27/34 → 26/34	61/88	25	늘 '다른 일' 46/89
같은 것, 먼 분야 묶음 28건 뺌	61	10 → 22 (16 % → 36 %)	—	54/61	25	묶음 28건의 손은 8/28
≤11-03 맞춤 / 11-04–05 시험	55	9 → 9 (16 % → 16 %)	25/30 → 23/30	50/55	22	늘 '잠깐' 39/55

이 시험에서 가장 많은 것을 알려 주는 것은 정해진 입력의 턴 배수다. 먼 분야 묶음이 맞추기에서 빠지면 ×0.234–0.465, 들어가면 ×0.117–0.234로 두 배 흔들린다. 그 묶음 28건은 모두 S_fixed_input = 예이고 턴의 가운데 값이 34로 적지만 분의 가운데 값은 14.8분으로 길다. 분당 턴이 2.41로 기준 띠(3.33–6.64)의 아래 끝보다 낮은데, 턴 하나하나가 큰 파일을 통째로 쓰는 쓰기라서다. 한 조각이 크기가 전혀 다른 일들을 한 숫자로 뭉치고 있고, '턴은 서로 비슷한 크기다'라는 뼈대의 전제(전제 1)가 이 묶음에서 깨진다. 실제로 이 묶음을 맞추기에 넣은 숫자(≤11-03)로 이 묶음 28건을 계산하면 가운데 값 7.5분으로 모두 '잠깐'이라 하여 28건 모두 틀리고, 넣지 않은 숫자(≤11-02)로는 가운데 값 14.0분으로 모두 '다른 일'이라 하여 28건 모두 맞는다 [사후]. 턴만 보면 거꾸로다. 넣은 숫자는 턴 가운데 38.1로 28건의 턴 범위를 모두 담고, 넣지 않은 숫자는 75.9로 하나도 담지 못한다. 턴을 맞힌 쪽이 분을 틀리는 것이다. 묶음 하나가 조각의 숫자와 성적표를 동시에 좌우하는 셈이다.

7.3 파일 수의 증거

파일 수 칸은 지금까지의 크기 칸 가운데 가장 깨끗하다. 메인이 실제 지시서에서 보내기 전에 세고, 단위를 파일 하나로 못 박았기 때문이다. 그러나 양이 적다. `files` 칸이 있는 기록 59건 중 빈 표(지시서가 수를 안 정함)가 18건, 결과 뒤에 채운 것이 11건이고, 보내기 전에 세어 0이 아닌 끝난 기록은 25건(exam_grader 12 · gemmin 11 · stock-rader 2)이다. 그중 정해진 일은 19건이고 저장소 둘(exam_grader 12, gemmin 7)에서만 나왔으며, exam_grader 12건은 한 에이전트의 연속 회차다. 이 19건에서 파일 수와 턴의 순위 상관은 +0.72, 분과는 +0.68로, 사후에 요약에서 센 `size`의 +0.21(2026-11-03, 맞추는 기록 88건)보다 훨씬 강하다. 저장소 안에서도 exam_grader +0.63, gemmin +0.75다. 하지만 파일 하나당 턴은 3–36(가운데 9)으로 열두 배 벌어진다 [사후]. 파일 수는 순서를 잘 맞히지만 단독으로 크기를 정하지는 못한다는 뜻이다.

> **물음: 정해진 일이고 일의 크기를 기록하는데도 왜 어려운가 — 데이터가 적어서인가?**
> 대체로 그렇다. 다만 '데이터가 적다'는 말은 두 겹으로 읽어야 한다. 첫째, 새 종류의 데이터, 곧 보내기 전에 지시서에서 센 파일 수는 끝난 정해진 일 19건, 저장소 둘, 사실상 에이전트 몇의 것뿐이다. 비율 조각의 시험 기록은 4건이다. 이 양으로는 '고정 몫 + 파일마다 몫'처럼 모수 둘인 모양도 저장소별로 안정되게 맞출 수 없다. 둘째, 지금 계산기의 구조는 그 수를 거의 쓰지 않는다. 스위치 경로(보낼 때 `calc`에 적는 답)는 상황 값과 저장소만 읽고 `files`를 아예 읽지 않으며, 양에 관한 정보는 사실상 S_fixed_input 스위치 하나(×0.036–0.066, batman 퍼즐 8건으로 맞춤)가 담는다. 비유 경로만 파일 수를 쓰는데, 비례로 쓰고 write·edit 비 중 큰 쪽을 골라 4건 모두에서 지나쳤다. 여기에 셋째 요인이 겹친다. 같은 '정해진 일' 안에서도 파일 하나의 비용(3–36턴)과 턴 하나의 길이(먼 분야 쓰기 턴 대 풀이지 편집 턴)가 크게 달라, 수를 센다고 크기가 닫히지 않는다. 그래서 숫자를 다시 맞추는 것만으로는 오르지 않고(16 % → 16 %), 늘어나는 기록이 같은 지시서 묶음이면 오른 것처럼 보이기만 한다.

---

8. 개입 지점

아래는 결정이 아니라 선택지다. 각 줄에 지금 있는 근거의 강도를 붙였다. 이 분야의 규칙상 구조를 바꿀 때는 까닭과 시험 방법을 먼저 적고 새 기록으로 시험해야 하며, 같은 시험 기록으로 고쳐 맞추면 사후 설명이 된다. 그러므로 표 12의 탐색은 표 13의 구조 후보를 '세우기 전에 적는' 재료를 얻는 일로 쓰는 것이 맞다.

표 12. 물어볼 만한 탐색 질문과 그 답을 줄 칸·명령.

#	질문	쓸 칸·명령	지금 아는 것
Q1	파일 하나의 턴 비용은 저장소·에이전트·write/edit에 따라 얼마나 다른가	`files`, `actual.turns`, `project`, `agent` (`size_data.py`)	정해진 일 19건 3–36턴(가운데 9). write와 edit를 따로 본 적은 없다
Q2	같은 에이전트 시리즈 안에서 턴 ≈ 고정 몫 + 파일마다 몫인가, 고정 몫은 얼마인가	`agent`, `round`, `files`, `actual` — exam_grader 12회차	비율 조각 4건이 모두 지나침 → 고정 몫 짐작과 맞음 (검증 안 됨)
Q3	턴 길이는 동작 섞임으로 설명되는가(전제 1)	`[dispatch.turns]` 표 74건 (동작마다 n·sum_s·turn_s)	쓰기·글 턴 약 20초, 읽기·명령 2–5초(분야 문서). 단 Bash 동작이 첫 낱말로 갈려 'Bash:cd'가 4,563턴 중 1,291턴 — 실제 동작이 가려진다(부록 A)
Q4	'모름'은 어디서 오나, 어떤 설정의 기준값이 먼저 필요한가	`model·effort`, `predict --all`	끝난 67건, 최근 55건 중 22건. 11-03 이후 모름 25건은 Haiku medium 12(gemmin) · Opus xhigh 7 · Opus medium 5 · Haiku 1
Q5	기간별 기준선(늘 같은 답)과 견주면 각 예측자는 얼마나 나은가	`date`, `actual.minutes`, `predicted.told`	11-04 이후 늘 '잠깐' 71 %, 손 91 %
Q6	같은 지시서 묶음을 하나로 셀 때 성적·상관이 어떻게 바뀌나	`agent`, `task`, `batch`	먼 분야 묶음 하나가 다시 맞추기 성적의 28/41을 만듦
Q7	'모름'이 '!예'를 통과해 걸린 조각의 성적은 따로 보면 어떤가	`situation`, `predict.matches`	적용 70회(C_turns_no_check_v2 34 등)
Q8	AI가 고른 빗댄 기록은 규칙이 고른 것과 무엇이 다른가(97 % 대 69 %)	`predicted.analog`, `analog <id>`	v1 시험에서 AI가 고른 기록이 규칙의 동점 묶음 안에 든 것 19/31
Q9	보낼 때 적은 `calc` 중 지금 조각의 스위치 경로와 같은 것은 몇 건인가	`calc`, git 이력의 `knowledge.toml`	p179(11-04): 같음 33 · 비유 22 · 다름 67(당시 122건)

표 13. 구조를 바꿀 자리. '근거'는 지금 있는 것의 강도.

#	바꿀 것	무엇이 달라지나	근거
S1	갈래 먼저, 그다음 파일마다: 턴 = T_0(갈래) + $n_{\text{write}}\cdot w + n_{\text{edit}}\cdot e$, 갈래 안에서 맞춤	고정 몫이 생겨 비율의 지나침이 사라질 수 있다. 갈래(저장소 + 일 종류, 또는 같은 에이전트 시리즈) 정의가 먼저 필요	순위 상관 +0.72(19건), 비율 4/4 지나침 사후
S2	약하게 거는 비율: $T = T_{\text{빗댄}} \times (n/n')^{\beta}$, $\beta < 1$	조각 하나만 고치면 된다. β를 미리 정하거나 새 기록으로 맞춤	제곱근이 4/4에서 비례보다 가깝고 2/4에서 곱하지 않음보다 가까움 사후, 4건
S3	스위치 경로가 수를 쓰게: S_fixed_input의 곱하기 하나를 입력 양·출력 양 숫자로(분야 문서의 후보 G8)	calc에 적는 답이 크기를 보게 된다. 지금 양 정보는 스위치 하나	정해진 입력 조각 시험 크기 1/63, 다시 맞추면 배수가 두 배씩 흔들림
S4	전제 1을 측정으로: 분당 턴을 지시서의 동작 섞임(쓰기·편집·읽기·명령 몇 번)으로 계산	'분당 턴' 조각이 조건이 아니라 계산값이 되고, 남는 불확실성이 양으로 모인다	동작별 턴 길이 열 배 차이(분야 문서). 측정 도구의 Bash 분류를 먼저 고쳐야 함
S5	'모름'의 처리: (가) 지금처럼 손으로 넘김, (나) 모델·effort별 기준값을 그 설정의 기록으로, (다) 다른 설정의 같은 일 × 모델 배수	최근 기록의 40 %가 계산기 밖에 있다	(나)는 Haiku medium 12건이 한 저장소. (다)는 R_models_*·C_min_light_model 성적이 약함(0/7)
S6	폭을 정직하게: 2배 자르기 대신 지난 '실측 ÷ 예측'의 분위수로 폭을 내고, 말할 구간과 폭을 따로 말함	크기 시험의 잣대(0.8)와 만드는 방식이 맞게 된다	285건 중 276건이 잘림. p179 A3
S7	시험 틀: 고정 홀드아웃 대신 날짜순 굴리는 시험 + 기준선 + 묶음 단위 빼기	'다시 맞추면 나아지나'를 매번 같은 틀로 잼	표 11처럼 묶음과 기준선이 결론을 바꿈
S8	'안 물음'과 '모름'을 다른 값으로, '!예'와 빗댄 기록의 와일드카드는 '안 물음'을 통과시키지 않음	조각이 조건을 확인하지 않은 채 걸리는 일이 없어짐	적용 70회. p179 G3
S9	빗댄 기록 v2 실패를 받아 v3를 세우기 전에 적을지, 규칙을 내릴지	미리 정한 기준으로 판정이 났다	규칙 22/32 대 AI 31/32 시험

표 14. 모을 칸. 이미 있는 칸 중 시끄러운 것도 함께 적었다.

#	칸	까닭	상태
D1	`files` 그대로 + 같은 지시서 묶음을 가리키는 묶음 id	묶음 단위 시험(Q6·S7)에 필요. 지금은 `agent`와 작업 설명 글로 짐작	files는 필수. 묶음 id는 없음
D2	지시서 길이(글자)·해시·경로	지시서 본문이 손이 이기는 정보원(6.1절)인데 남지 않는다	없음. 지난 기록은 채울 수 없음
D3	`calc`에 경로(스위치/비유)와 조각 판(seen_until·커밋)	'손 vs 계산기'를 같은 계산기끼리 셀 수 있다	없음(p179 G1)
D4	입력 양(쪽·KB)·출력 양 숫자	S3의 재료	없음(G8)
D5	턴 표의 Bash 동작을 실제 명령(cd 다음 낱말)으로	Q3·S4의 재료	측정 도구 수정 필요
시끄러움	`size`(단위 섞임, 요약에서 사후 채움), S_full_brief(333/334 예), S_similar_past(주관), `predicted.analog`(AI 선택, 재현 안 됨), 옛 `calc`(판 섞임)	설명 변수로 쓸 때 조심	—

부록 A. 문서와 코드가 어긋난 곳

2026-11-05에 코드·명령 출력과 분야 문서(`domains/agent_usage/CLAUDE.md`)를 대조해 찾은 것이다. 고치지는 않았다.

1. '지금 상태'의 "기록 292건" — 지금 362건(끝난 352). 같은 절의 "audit 오류 0, 경고 22(2026-11-02)" — 지금 오류 1, 경고 24.
2. 홀드아웃 뒤 절반을 문서는 61건으로 적고, `dispatches.py` 587행은 "뒤 절반 61건: 손 62 %, 계산기 51 %"를 글자로 박아 출력한다. `holdout.py`의 실제 출력은 62건, 손 63 %, 쪼갠 뒤 계산기 50 %다.
3. 빗댄 기록 규칙 v2를 문서는 '시험 전(새 기록 30건)'으로 적지만, `analog` 출력은 시험 기록 32건, 규칙 69 % 대 AI 97 %로 '실패'를 낸다.
4. 크기 모델의 첫 시험 숫자(33건)는 11-03 것이고, 지금 출력은 시험 기록 65건이다(나눈 모델 1/65, 안 나눈 모델 15/65).
5. '계산 조각 16개 … 이전 버전 5개' — 계산 조각 구역의 이전 버전은 6개다(C_turns_research, C_turns_no_check, C_turns_resumed, C_fix_existing, C_small_codebase, C_turns_spec_fixed).
6. `holdout.piece_scores`가 비유 효과를 건너뛰어 C_turns_analog는 성적표에서 영원히 '시험 전'이다. 문서는 '비유 조각: 성적을 따로 낸다'고 하지만, 따로 내는 곳은 `holdout` 출력 끝의 '맞춰 본 것' 목록뿐이다.
7. `holdout._scale_scores`는 이번 기록의 `files.after`만 보고 빗댄 기록의 `after`는 보지 않는다(p189 → p185).
8. `scripts/agent_usage.py --turns`는 Bash 동작을 명령의 첫 낱말로 가르는데, 명령 대부분이 `cd …&&`로 시작해 턴 표 74건의 4,563턴 중 1,291턴이 'Bash:cd'로 뭉친다. 전제 1 측정의 해상도가 여기서 깎인다.
9. S_many_turn_model의 기본값 '아니오' 때문에 Opus xhigh와 Haiku의 정해진 입력 풀이도 '턴을 잘게 쓰지 않는 모델'(C_min_light_model)로 들어간다. 다시 맞추면 그 효과가 ×0.086–0.16으로 무너진다(표 7).
10. '!예' 조건을 '모름'이 통과한다(코드의 뜻). 분야 문서에는 적혀 있지 않고 p179 보고서에만 있다.
11. '지금 쓰는 규칙 요약'의 "맞춰 본 것만 83/125", "51 % vs 62 %"는 10-19 숫자다.

부록 B. 용어 정리

용어	뜻
보냄 · 회차 · 배치	지시서 하나로 하위 에이전트에 맡긴 일(`[[dispatch]]` 한 건) · 같은 에이전트에 이어 보낸 몇 번째 일 · 함께 내보낸 보냄의 묶음
턴(= 블록)	대화 기록의 모델 줄 하나. 응답 하나가 생각·글·도구 호출 블록마다 여러 줄로 남으므로 응답 수가 아니다
시간 구간 · 말할 구간	잠깐(≤10분)·다른 일(10–40)·신경 끄기(>40) · 예측 분 범위의 기하 가운데 값이 든 구간(R_say_numbers_v3)

용어	뜻
상태 · 스위치 · 대 보기	보내기 전에 지시서로 답하는 예/아니오/일부/모름 질문 · 조각의 조건으로 쓰일 때의 상태 · 기록에 상태마다 값을 적는 일
known_before	보내기 전에 알 수 있는 상태인가. 거짓이면 계산 조건으로 못 쓴다
조각 · 효과	효과 칸이 붙은 살아 있는 가설 · 무엇에·어떻게(기준/곱하기/더하기/비율)·범위·언제·방향의 한 묶음
스위치 경로 · 비유 경로	상태와 저장소만으로 기준값 × 곱하기를 계산하는 길 · 보내기 전에 적은 빗댄 기록의 실측에서 출발하는 길
빗댄 기록	이번 일과 같은 모양이라고 보고 고른 지난 기록. 손이 고른 것(`predicted.analog`)과 규칙이 고른 것(`analog.py`)이 있다
비율 조각	범위 숫자 없이 '이번 수 ÷ 빗댄 기록의 수'를 배수로 곱하는 조각(C_turns_files_scale)
정해진 일 · 찾는 일	만들 것과 고칠 곳이 지시서에 다 적힌 일(S_specified = 예) · 찾을 것의 수가 지시서에 없는 일(조사·원인 찾기)
손 예측 · 계산기 답	메인이 보내기 전에 적은 예측(`predicted`) · 보낼 때 돌린 계산기의 답(`calc`)
seen_until · 맞춰 본 것 · 시험	조각을 세우거나 맞출 때 본 마지막 기록 · 그때까지의 기록으로 잰 성적 · 그 뒤 기록으로 잰 성적
창(window)	값들을 훑어 상한/하한 ≤ 2로 가장 많이 담는 구간. 범위 숫자를 정하는 방법
방향 시험 · 크기 시험	조건 있음·없음의 가운데 값 비교가 주장한 방향과 같은가 · 걸린 기록 중 범위 안 몫이 0.8 이상인가
쪼개기 · 이전 버전	숫자를 넓히는 대신 차이를 만드는 조건을 더해 조각을 나누는 일 · 그때 남기는 옛 조각(`status = "retired"`)
조건 흐림 · 이유 틀림 · 바깥 상황 · 까닭 모름	돌아보기의 까닭 넷: 조건을 더해 쪼갠다 · 새 버전으로 고친다 · 예외로 둔다(시험에서 뺌) · 고치지 않고 쌓아 사용자에게 넘긴다
전제 1	'턴은 서로 비슷한 크기다'. 시간 = 턴 ÷ 분당 턴이라는 뼈대의 가정. 측정(턴 길이 표)으로 치우기로 한 것
audit · 장치	그래프와 성적에서 기계로 보는 점검(오류·경고) · 코드와 테스트가 막는 규칙(A–H)
after	`files`를 결과를 본 뒤에 채웠다는 표시. 비율 조각 시험에서 뺀다